ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ _____

ПРЕПОДАВАТЕЛЬ

канд. техн. наук , доцент		А.В. Бржезовский
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

СОЗДАНИЕ ТАБЛИЦ БАЗЫ ДАННЫХ

по курсу: Методы и средства проектирования информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4329		Д.С. Шаповалова
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1. Цель работы:

Произвести создание БД и таблиц, определив правила проверки значений и задав ограничения ссылочной целостности.

2. Вариант задания - 16

16. Создайте базу данных для хранения следующих сведений: город, район, квартира, комната, площадь, цена, агент, продажа. Составьте запросы, позволяющие выбрать:

а) перечень однокомнатных квартир, продаваемых в Московском районе;

б) квартиры, находящиеся на одной улице, но в различных районах;

в) двух- и трехкомнатные квартиры, имеющие одинаковую площадь;

г) средняя цена однокомнатной квартиры в городе;

д) районы, в которых продается наибольшее число объектов недвижимости;

е) районы, в которых минимальна стоимость квадратного метра;

ж) улицы, продолжительность которых ограничивается только одним районом;

з) районы, в которых не продаются однокомнатные квартиры;

и) районы, в которых продаются квартиры всех строительных серий.

3. Схема БД.

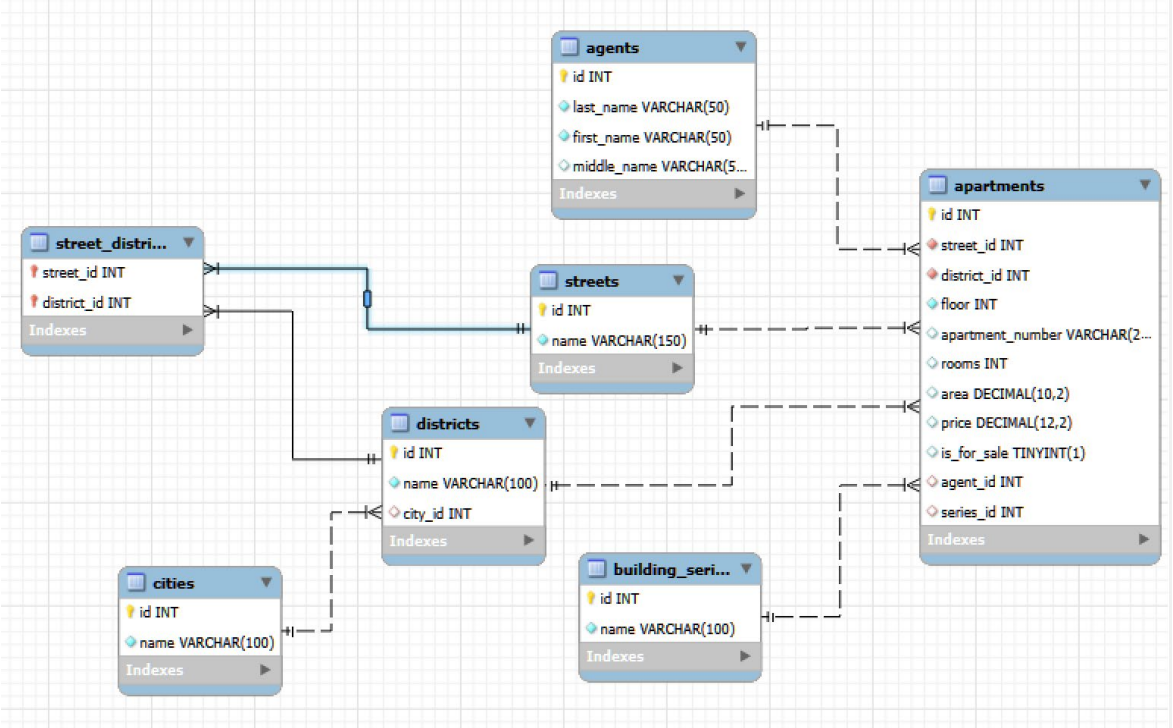

Рисунок 1 – Диаграмма БД.

4. Скрипт SQL:

```
USE real_estate;

-- 1. Города
CREATE TABLE cities (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   CONSTRAINT chk_city_name CHECK (CHAR_LENGTH(TRIM(name)) > 0)
);

-- 2. Районы
CREATE TABLE districts (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   city_id INT DEFAULT NULL,
   CONSTRAINT chk_district_name CHECK (CHAR_LENGTH(TRIM(name)) > 0),
   CONSTRAINT fk_district_city
      FOREIGN KEY (city_id)
      REFERENCES cities(id)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);

-- 3. Улицы
CREATE TABLE streets (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(150) NOT NULL,
   CONSTRAINT chk_street_name CHECK (CHAR_LENGTH(TRIM(name)) > 0)
);

-- 4. Связь Улицы-Районы
CREATE TABLE street_districts (
   street_id INT NOT NULL,
   district_id INT NOT NULL,
   PRIMARY KEY (street_id, district_id),
```

```
    CONSTRAINT fk_sd_street
        FOREIGN KEY (street_id)
        REFERENCES streets(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    CONSTRAINT fk_sd_district
        FOREIGN KEY (district_id)
        REFERENCES districts(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 5. Агенты
CREATE TABLE agents (
    id INT PRIMARY KEY AUTO_INCREMENT,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    middle_name VARCHAR(50) DEFAULT NULL,
    CONSTRAINT chk_agent_names CHECK (
        CHAR_LENGTH(TRIM(last_name)) > 0 AND
        CHAR_LENGTH(TRIM(first_name)) > 0
    )
);

-- 6. Строительные серии
CREATE TABLE building_series (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    CONSTRAINT chk_series_name CHECK (CHAR_LENGTH(TRIM(name)) > 0)
);

-- 7. Квартиры
CREATE TABLE apartments (
```

```
id INT PRIMARY KEY AUTO_INCREMENT,
street_id INT NOT NULL,
district_id INT NOT NULL,
floor INT NOT NULL DEFAULT 1,
apartment_number VARCHAR(20) DEFAULT NULL,
rooms INT DEFAULT NULL,
area DECIMAL(10, 2) DEFAULT NULL,
price DECIMAL(12, 2) DEFAULT NULL,
is_for_sale BOOLEAN DEFAULT TRUE,
agent_id INT DEFAULT NULL,
series_id INT DEFAULT NULL,

-- CHECK ограничения для числовых данных
CONSTRAINT chk_floor CHECK (floor > 0),
CONSTRAINT chk_rooms CHECK (rooms IS NULL OR rooms > 0),
CONSTRAINT chk_area CHECK (area IS NULL OR area > 0),
CONSTRAINT chk_price CHECK (price IS NULL OR price >= 0),

-- Внешние ключи с каскадным удалением
CONSTRAINT fk_apt_street
   FOREIGN KEY (street_id)
   REFERENCES streets(id)
   ON DELETE CASCADE
   ON UPDATE CASCADE,

CONSTRAINT fk_apt_district
   FOREIGN KEY (district_id)
   REFERENCES districts(id)
   ON DELETE CASCADE
   ON UPDATE CASCADE,

CONSTRAINT fk_apt_agent
   FOREIGN KEY (agent_id)
   REFERENCES agents(id)
   ON DELETE CASCADE
```

```
        ON UPDATE CASCADE,

    CONSTRAINT fk_apt_series
        FOREIGN KEY (series_id)
        REFERENCES building_series(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

5. Описание структуры

1) Ограничения на значения данных (Целостность домена)

Для обеспечения корректности вводимой информации на уровне полей установлены следующие ограничения:

1. Первичные ключи (PRIMARY KEY):
   - Во всех таблицах поле id является уникальным идентификатором записи.
   - Для всех полей id установлен атрибут AUTO_INCREMENT, что гарантирует уникальность и автоматическую генерацию идентификаторов при вставке новых записей.
   - В таблице street_districts первичный ключ составной (street_id, district_id), что запрещает дублирование связей между одной и той же парой улицы и района.
2. Обязательные поля (NOT NULL):
   - Текстовые названия (name в справочниках, фамилия и имя агента) не могут быть пустыми.
   - Географические привязки квартиры (street_id, district_id) и этаж (floor) являются обязательными.
   - Числовые параметры (rooms, area, price) могут принимать значение NULL (если данные неизвестны), за исключением этажа.
3. Типы данных:
   - Денежные суммы и площадь: Использован тип DECIMAL (например, DECIMAL(10, 2) для площади и DECIMAL(12, 2) для цены). Это исключает ошибки округления, характерные для типа FLOAT.
   - Логические значения: Статус продажи is_for_sale реализован через тип BOOLEAN (в MySQL хранится как TINYINT(1)), по умолчанию имеет значение TRUE.
   - Текстовые поля: Использован тип VARCHAR с ограничением длины для оптимизации хранения (например, VARCHAR(50) для фамилии).
4. Уникальность (UNIQUE):
   - Таблица building_series: Поле name имеет ограничение UNIQUE. Это запрещает создание дубликатов строительных серий (например, нельзя добавить серию «П-44» дважды).
5. Проверочные ограничения (CHECK):

Для валидации данных на уровне базы установлены следующие правила (требуют MySQL 8.0.16+):

- Текстовые поля (cities, districts, streets, agents, building_series): Запрещены пустые строки или строки из пробелов (CHAR_LENGTH(TRIM(name)) > 0).
- Квартиры (apartments):
  1. Этаж (floor): должен быть больше 0.
  2. Комнаты (rooms), Площадь (area): должны быть больше 0 (если значение не NULL).
  3. Цена (price): должна быть >= 0 (если значение не NULL).

2) Ссылочная целостность (Referential Integrity)

Связи между таблицами реализованы через внешние ключи (FOREIGN KEY). Для обеспечения согласованности данных при удалении записей настроено каскадное удаление (ON DELETE CASCADE).

2.1) Логика каскадного удаления

- Удаление в старшей (родительской) таблице: Если удаляется запись из справочника (например, удаляется Агент), то все зависимые записи в дочерней таблице (Квартиры, где указан этот агент) удаляются автоматически.
  - *Пример:* Удаление Города → Удаляются Районы → Удаляются связи Улиц с районами → Удаляются Квартиры в этих районах.
- Удаление в младшей (дочерней) таблице: Если удаляется запись из дочерней таблицы (например, удаляется Квартира), это не влияет на записи в справочниках (Агенты, Улицы, Районы остаются в базе).

2.2) Схема связей

1. Города → Районы:
   - districts.city_id ссылается на cities.id.
   - При удалении города все его районы удаляются.
2. Районы ↔ Улицы (Многие-ко-многим):
   - Реализовано через таблицу street_districts.
   - street_districts.street_id ссылается на streets.id.
   - street_districts.district_id ссылается на districts.id.
   - При удалении улицы или района соответствующие записи связи удаляются.
3. Справочники → Квартиры:
   - Таблица apartments имеет внешние ключи на streets, districts, agents, building_series.
   - При удалении улицы, района, агента или строительной серии все квартиры, ссылающиеся на них, удаляются из реестра.

3. Особенности реализации

- Гибкая географическая привязка: Поскольку одна улица может проходить через несколько районов, прямая ссылка district_id в таблице streets была исключена. Вместо этого используется таблица-связка street_districts. Однако в таблице apartments хранятся конкретные street_id и district_id, что позволяет точно идентифицировать местоположение конкретного объекта недвижимости в рамках пересечения улицы и района.
- Валидация на уровне БД: Основные бизнес-правила (цена не отрицательная, этаж положительный, уникальность серий) реализованы средствами СУБД (CHECK, UNIQUE), что защищает данные от ошибок даже при вставке через сторонние скрипты.

Вывод

В ходе выполнения работы была спроектирована и реализована структура реляционной базы данных для системы учета недвижимости «real_estate» на основе СУБД MySQL.

В результате были решены следующие задачи:

1. **Нормализация данных:** Информация разделена на логические таблицы (справочники городов, районов, улиц, агентов и реестр квартир), что устраняет избыточность данных и упрощает поддержку актуальности информации.
2. **Обеспечение целостности:** Реализована ссылочная целостность через внешние ключи с каскадным удалением (ON DELETE CASCADE), а также целостность домена через строгие типы данных и ограничения NOT NULL.
3. **Валидация данных:** Внедрены ограничения CHECK и UNIQUE для предотвращения ввода некорректной информации (отрицательная цена или площадь, пустые имена, дубликаты строительных серий) на уровне СУБД.
4. **Автоматизация:** Настроены значения по умолчанию (DEFAULT) и автоинкремент идентификаторов (AUTO_INCREMENT) для упрощения вставки записей и стандартизации данных (например, статус продажи по умолчанию).

Особое внимание уделено гибкости структуры: связь «многие-ко-многим» между улицами и районами реализована через промежуточную таблицу street_districts, что позволяет точно описывать географическое положение объектов, где одна улица может проходить через несколько районов.

Итоговая схема базы данных является масштабируемой, защищенной от ошибок ввода и готова к наполнению данными и интеграции с прикладным программным обеспечением.